

QMS Raspberry Pi Deployment Flow Guide

GitHub-based remote software deployment for the MQTT Queue Manager

Repository: sonvisageabuja1/QMS_Pi_code

Main application: /home/sonvisage/QMS_Pi_code/mqtt_queue_manager.py

Runtime service: mqtt_queue.service

Update timer: qms-update.timer

Update interval: every 5 minutes

DEVELOPMENT	GITHUB	REMOTE RASPBERRY PI
Edit and test QMS code	main branch	qms-update.timer
git add / commit / push	New commit	Checks every 5 minutes
		git fetch + fast-forward
		Restart mqtt_queue.service
		New QMS version running

1. Current QMS Layout

The deployed program is kept inside a dedicated Git repository. The Python virtual environment remains outside the repository.

```
/home/sonvisage/  
|-- mqtt-env/  
`-- QMS_Pi_code/  
    |-- mqtt_queue_manager.py  
    |-- rfid_mappings.txt  
    |-- esp32_mqtt_listener.py  
    |-- mqtt_logger.py  
    |-- queues.json  
    `-- .gitignore
```

Important: queues.json is runtime data and is ignored by Git. A software update therefore does not replace the live queue state. Historical mqtt_queue_manager version files are also ignored.

2. GitHub Repository

```
git@github.com:sonvisageabuja1/QMS_Pi_code.git  
Branch: main
```

The development Pi uses SSH authentication. The private key stays on the Raspberry Pi and must never be copied into the repository.

3. MQTT Queue Manager systemd Service

File: /etc/systemd/system/mqtt_queue.service

```
[Unit]  
Description=MQTT Queue Manager Service  
After=network-online.target  
Wants=network-online.target  
  
[Service]  
Type=simple
```

```
User=sonvisage
WorkingDirectory=/home/sonvisage/QMS_Pi_code
ExecStart=/home/sonvisage/mqtt-env/bin/python /home/sonvisage/QMS_Pi_code/mqtt_queue_manager.py
Restart=always
RestartSec=5
Environment=PYTHONUNBUFFERED=1

[Install]
WantedBy=multi-user.target
```

Service commands

```
sudo systemctl daemon-reload
sudo systemctl enable mqtt_queue.service
sudo systemctl restart mqtt_queue.service
systemctl status mqtt_queue.service --no-pager
journalctl -u mqtt_queue.service -n 50 --no-pager
```

4. Automatic Update Components

The timer checks GitHub every five minutes. No new commit means no restart. A new commit is fetched and fast-forwarded, then `mqtt_queue.service` is restarted.

Updater script: `/usr/local/bin/qms-update.sh`

```
#!/bin/bash
set -u

REPO="/home/sonvisage/QMS_Pi_code"
BRANCH="main"
SERVICE="mqtt_queue.service"
LOG_TAG="qms-updater"

log() {
  logger -t "$LOG_TAG" "$1"
  echo "$1"
}

log "Checking GitHub for QMS updates..."

if ! sudo -u sonvisage \
HOME=/home/sonvisage \
GIT_SSH_COMMAND="/usr/bin/ssh -i /home/sonvisage/.ssh/id_ed25519 -o IdentitiesOnly=yes" \
/usr/bin/git -C "$REPO" fetch origin "$BRANCH"; then
  log "ERROR: Unable to contact GitHub. Existing QMS remains running."
  exit 1
fi

LOCAL=$(sudo -u sonvisage /usr/bin/git -C "$REPO" rev-parse HEAD)
REMOTE=$(sudo -u sonvisage /usr/bin/git -C "$REPO" rev-parse "origin/$BRANCH")

if [ "$LOCAL" = "$REMOTE" ]; then
  log "QMS is already up to date."
  exit 0
fi

log "New QMS version detected: $LOCAL -> $REMOTE"

if ! sudo -u sonvisage \
HOME=/home/sonvisage \
/usr/bin/git -C "$REPO" merge --ff-only "origin/$BRANCH"; then
  log "ERROR: Update could not be fast-forwarded. Existing QMS remains running."
  exit 1
fi

log "QMS code updated successfully."

if /usr/bin/systemctl restart "$SERVICE"; then
  log "QMS service restarted successfully."
else
  log "ERROR: Code updated but QMS service failed to restart."
  exit 1
fi
```

Update service: `/etc/systemd/system/qms-update.service`

```
[Unit]
Description=QMS GitHub Automatic Update
After=network-online.target
Wants=network-online.target

[Service]
Type=oneshot
User=root
ExecStart=/usr/local/bin/qms-update.sh
```

Update timer: `/etc/systemd/system/qms-update.timer`

```
[Unit]
Description=Check GitHub for QMS updates every 5 minutes

[Timer]
```

```
OnBootSec=2min
OnUnitActiveSec=5min
Persistent=true
Unit=qms-update.service
```

```
[Install]
WantedBy=timers.target
```

Enable and verify the timer

```
sudo systemctl daemon-reload
sudo systemctl enable --now qms-update.timer
systemctl status qms-update.timer --no-pager
systemctl list-timers --all | grep qms
```

5. Normal Software Release Procedure

Edit and test the program on the development Pi before publishing it.

```
cd /home/sonvisage/QMS_Pi_code
nano mqtt_queue_manager.py

sudo systemctl restart mqtt_queue.service
systemctl status mqtt_queue.service --no-pager
```

When the new version is confirmed working:

```
cd /home/sonvisage/QMS_Pi_code
git status
git add mqtt_queue_manager.py
git commit -m "Describe the QMS update"
git push origin main
```

Deployment Pis will detect the new main-branch commit within approximately five minutes, update, and restart the MQTT Queue Manager.

6. RFID Mapping Updates

```
cd /home/sonvisage/QMS_Pi_code
git add rfid_mappings.txt
git commit -m "Update RFID mappings"
git push origin main
```

7. Manual Update and Diagnostic Commands

```
# Force an immediate update check
sudo /usr/local/bin/qms-update.sh

# QMS status
systemctl status mqtt_queue.service --no-pager

# QMS logs
journalctl -u mqtt_queue.service -n 50 --no-pager

# Timer status
systemctl status qms-update.timer --no-pager

# Updater logs
journalctl -u qms-update.service -n 50 --no-pager

# Installed Git version
git -C /home/sonvisage/QMS_Pi_code log -1 --oneline

# Repository state
git -C /home/sonvisage/QMS_Pi_code status
```

8. Safety Behavior

If the internet or GitHub is unavailable, the existing QMS remains running. If there is no new commit, the service is not restarted. Updates use fast-forward-only Git merges. Git operations run as sonvisage so repository ownership remains correct.

9. Recommended Multi-Pi Deployment Model

Use one development/master Pi to edit, test, commit, and push releases. Deployment Pis should normally receive read-only repository access and automatically pull approved releases. This reduces the risk of a field Pi accidentally pushing changes to GitHub.